



Shares



Arduino

[Arduino Uno](#)[Arduino Mega](#) [Arduino Nano](#)[Proyectos Arduino](#) [Programar con Arduino](#)[Tipos de Arduino](#) [Los 11 mejores accesorios para Arduino](#)[Los Mejores Kits de Inicio de Arduino](#)[Arduino IDE](#)[Domótica y Arduino](#)[10 proyectos Arduino](#)[Cómo aprender robótica con Arduino](#)[Raspberry Pi 4](#)[Raspberry Pi, breve guía, modelos y características](#)[5 maneras de escritorio remoto en Raspberry Pi \(Windows/Linux/Mac\)](#)[Raspberry Pi Pico, análisis e introducción](#)[Las mejores alternativas a la Raspberry Pi](#)[Educación](#)[Programación para niños ¿Por qué debemos enseñar a los niños a programar?](#)[Scratch, un lenguaje fácil de Programación para los niños](#)[Robótica para niños, los mejores robots para iniciar a los peques en la robótica](#)[Robótica](#)[¿Cómo Programar un Robot? Lenguajes para Programar Robots](#)[Robótica Educativa](#)[Robots Industriales](#)[Robots colaborativos](#)[Aprender robótica con ROS](#)[Automatización industrial](#)[Electrónica](#)[NodeMCU](#)[ESP8266](#)[Mosfet](#)[LM317](#)[Dht11](#)[Protoboard](#)[MQTT](#)[ESP32 vs ESP8266 ¿Cuales son las diferencias entre ambos módulos?](#)[Impresión 3D](#)[Filamentos de impresión en 3D](#)[Los mejores filamentos para impresoras 3D](#)[Impresión 3D y Robótica](#)[Las Mejores impresoras 3D](#)[Filamento PVA](#)[Impresión 3D, guía para aprender y empezar](#)[Materiales de impresión en 3D](#)[Cómo ganar dinero con una impresora 3D](#)[Placas de circuito impreso en 3D y las impresoras 3D de PCB](#)[Los 10 mejores escáneres 3D](#)[10 consejos para impresión con resina con impresoras 3D SLA](#)[Filamento ABS](#)[Filamento PLA](#)

[Anycubic i3 MEGA Impresora 3d, lo mejor para imprimir en casa](#)

[Ender 5: ¿comprar o no? ¡Descúbrelo aquí!](#)

[Las 5 impresoras 3D más rápidas 2021 de todos los precios](#)

[Las 5 mejores impresoras 3D por debajo de 300 €](#)

[Top Las Mejores impresoras 3D de 2021 para todos los precios](#)

Shares



ARDUINO - GENUINO / ESPECIALES

POR [REDACCIÓN](#) · PUBLICADA · ACTUALIZADO

Arduino para dummies, una guía básica para principiantes



BOLETÍN

☒ Suscribirse a nuestro Boletín

☐ Desuscribirse

Dirección de Correo:

BUSCAR

ULTIMAS ENTRADAS

🕒 [Iniciar Proyectos IoT con ESP32: Guía Completa Paso a Paso](#)

🕒 [ESP32 vs Arduino: Diferencias Clave, Características y Guía para Elegir el Mejor Microcontrolador](#)

🕒 [ESP32-C3: El Microcontrolador que Redefine el IoT Sencillo y Seguro](#)

🕒 [Como medir la temperatura y la presión barométrica con Arduino y BMP280](#)

🕒 [El Cerebro de tu IoT: ¿Qué Broker MQTT Impulsará tu Proyecto?](#)

Tabla de Contenidos



1. Cómo funciona Arduino, Arduino desde Cero, Primeros Pasos con Arduino
2. Introducción, definición de Arduino
 - 2.1. Microcontrolador
 - 2.2. Entradas o Inputs
 - 2.3. Salidas o Outputs
3. Tipos de placas
4. Programar arduino desde cero, El bootloader o la vida de tu programa
5. Software, programar Arduino, el IDE
6. Cable USB
7. Programación de Arduino
 - 7.1. Creación del proyecto
8. Funciones
 - 8.1. Función de gestión de E/S digitales
 - 8.2. Función de gestión de E/S analógicas
 - 8.3. Funciones de temporización
9. Función Scheduler
 - 9.1. Funciones personalizadas
 - 9.2. Monitor en serie
10. Librerías Arduino
11. Pasos para configurar la placa
12. Shields o módulos
13. Componentes y Accesorios
14. Enlaces de interes

Shares



Esta es una guía de Arduino básica para aquel que comienza, queremos que sea un **tutorial arduino** que sirva de referencia a la hora de iniciarse en Arduino, es importante para comenzar a usar **Arduino desde Cero**, al final del artículo os añadiré unos enlaces interesantes para principiantes también de nuestra web. Si aún no sabes lo que es Arduino pásate por este enlace antes. [¿Que es Arduino?](#)

Introducción, definición de Arduino

Como hemos dicho en artículos anteriores y ya muchos sabéis, [Arduino](#) es un **hardware basado en el código abierto**, es un microcontrolador que se puede utilizar para muchas aplicaciones diferentes. En resumidas cuentas Arduino es una plataforma de prototipado electrónico de código abierto basada en [hardware](#) y software flexible y fácil de usar. Está dirigido a artistas, diseñadores, aficionados y cualquier persona interesada en crear objetos o entornos interactivos. Arduino es una computadora pequeña que se puede programar para leer y controlar componentes eléctricos conectados a ella.

Podría decirse que es la opción más fácil, con más comunidad detrás y menos costosa entre los microcontroladores para aficionados, estudiantes y profesionales para desarrollar proyectos basados en microcontroladores. Si aún no te está decidido por Arduino échale un vistazo a [estas razones](#) que tienes para decidirte por esta plataforma.

Las placas Arduino utilizan chips Atmel AVR o Atmel ARM , y algunas versiones tienen una interfaz USB. También tienen seis o más [pines](#) de entrada analógicas y catorce o más entradas digitales / salidas (I / O)

pinos que se utilizan para conectar sensores, módulos o shields y otros circuitos periféricos al microcontrolador. Además una placa de este tipo no es muy cara, si la original te parece cara siempre tienes clones.

Shares

Microcontrolador



Arduino es un microcontrolador en una tarjeta de circuito que facilita la recepción de entradas y salidas de accionamiento. Un microcontrolador es una computadora integrada en un chip.



Entradas o Inputs



Algunos ejemplos de entradas serían un sensor de temperatura, un sensor de movimiento, un sensor de distancia, un interruptor, etc.



Salidas o Outputs

Algunos ejemplos de salidas serían una luz, una pantalla, un motor, etcétera.

Tipos de placas



Hay muchos tipos diferentes de placas disponibles, como se muestra en la siguiente lista, cada uno con su propio conjunto de características. Se diferencian en cuanto a la velocidad de procesamiento, memoria, puertos I / O, y la conectividad, pero su funcionalidad básica es la misma. Las siguientes son algunas de ellas.

- [Arduino Uno](#)
- [Arduino MKR1000](#)
- Arduino Leonardo
- Arduino Due
- Arduino Yun
- Arduino Tre
- Arduino Micro
- Arduino Robot
- Arduino Esplora
- [Arduino Mega](#)
- Arduino Mini
- Arduino LilyPad
- Arduino Nano
- Arduino Fio

■ Arduino Pro

Tienes un resumen de las placas más importantes y usadas en el siguiente artículo de nuestra web, [elegir la placa Arduino adecuada para tu proyecto](#).

Shares

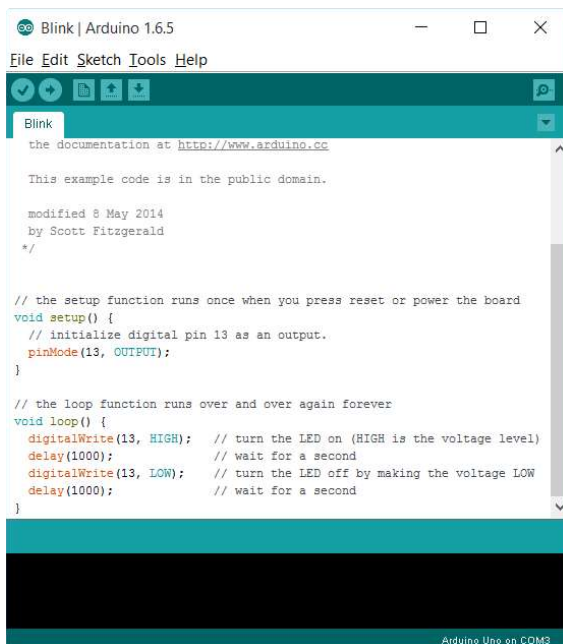
Programar arduino desde cero, El bootloader o la vida de tu programa

La memoria del microcontrolador que llevan estas placas es del tipo Flash, es decir que persiste aunque se le desconecte de la fuente de alimentación. Para cambiar el programa dentro de la placa, se necesita otro programa, que reside en la parte protegida de la placa. Ese programa se llama bootloader.

El bootloader es un programa que ya está precargado en la placa y que permite volver a programarlo desde un ordenador. Una vez se carga un programa nuevo en la placa, el programa permanece ahí hasta que se cargue uno nuevo o una nueva versión.

Software, programar Arduino, el IDE

El **software utilizado para programar la placa** se denomina “**entorno de desarrollo integrado**” (IDE). El IDE es una aplicación Java que se ejecuta en muchas plataformas diferentes, incluyendo PC, Mac y Linux. Está desarrollado para los principiantes que no están familiarizados con la programación. Incluye un editor de código, un compilador y un cargador.



También se incluyen las bibliotecas de código para el uso de periféricos, tales como puertos serial y varios tipos de pantallas. Los programas realizados para Arduino son llamados “sketches” o «bocetos», y están escritos en un lenguaje muy similar a C o C++. El software es libre y gratuito.

Cable USB

Shares



La mayoría de placas se pueden conectar a un equipo host a través de un cable USB. Esta conexión permite subir programas a la placa, así como proporcionar alimentación a la placa.

Programación de Arduino



El lenguaje de programación de Arduino está basado en los lenguajes C y C++. Si ya los conoces, no tendrás problemas para familiarizarte con el entorno de Arduino.

Si eres principiante, es mejor usar el software oficial de Arduino. Puede descargarse gratuitamente de Internet (en esta dirección: <https://www.arduino.cc/en/Main/Software>). Sin embargo, debes tener en cuenta que hay muchas alternativas viables para programar tu Arduino desde otro software.

La programación de un Arduino es fácil: como he dicho un poco más arriba, se utiliza el editor de código IDE para escribir el programa y luego compilarlo y cargarlo con un solo clic.

Un programa incluye dos funciones principales:

- `setup()` La función `setup()` se utiliza para inicializar la configuración de la placa. Esta función se ejecuta sólo una vez, cuando el tablero se enciende.
- `loop()` La función `loop()` se ejecuta después de la `setup()` completa, y a diferencia de la función `setup()`, se ejecuta continuamente.

Funciones de programación. Estas son algunas de las funciones más utilizadas en la programación de Arduino: (los veremos más detenidamente un poco más abajo)

- `pinMode` – establece el modo de pin de entrada o salida.
- `analogRead` – lee un voltaje analógico de un pin de entrada analógica.
- `analogWrite` – escribe un voltaje analógico a un pin de salida analógica.
- `digitalRead` – lee el valor de un pin de entrada digital.
- `digitalWrite` – establece el valor de un pin de salida digital para alta o baja.
- `Serial.print` – datos grabados en el puerto serie como texto ASCII legible.

Si se te hace difícil programar a través del IDE de Arduino hay también alternativas gráficas para poder hacerlo, tienes las más interesantes en [este artículo de nuestra web, donde vemos 5 alternativas gráficas para](#)

[programar nuestro Arduino.](#)

Creación del proyecto

Shares



Una vez que el software ha sido descargado e instalado, puedes crear tu primer proyecto. Simplemente haz clic en Archivo, luego en Nuevo, o utiliza los atajos de teclado ctrl+N. Al crear un proyecto, esto es lo que obtienes:



Se puede ver que se generan automáticamente dos funciones. Ya las hemos visto un poco más arriba pero lo volvemos a ver:

- **Setup():** La función de setup es una función que se ejecuta sólo una vez cada vez que se inicia el programa. En esta función se inicializan las variables, se define la función de cada cabezal (entrada o salida), la inicialización del estado de las salidas, las funciones de inicialización personalizadas o cualquier otra parte del código que se quiera ejecutar una sola vez.
- **Loop():** la función loop es la función principal de Arduino: una vez ejecutada la función setup, la función loop se ejecutará y repetirá una y otra vez.

El software es muy fácil de usar. En la barra de iconos (visible en la parte superior de la imagen anterior), puedes ver cinco botones en la parte superior izquierda. La primera (comprobar) se utiliza para compilar el código y comprobar que no hay errores en él. El segundo (upload) se usa para enviar el programa al Arduino para su ejecución. Los tres iconos restantes se utilizan para crear (nuevo), abrir o guardar un proyecto.

El icono en la parte superior derecha se utiliza para abrir el monitor serie. El uso de éste se detallará a continuación.

Otra cosa muy importante que debes saber es que **cuando conectas una tarjeta USB, tienes que ir al menú «tools» y seleccionar el tipo de tarjeta a la que quieres transferir tu proyecto y el puerto COM al que está conectado el Arduino**. Es necesario hacer esta manipulación cada vez para que la tarjeta sea reconocida por el software de Arduino y la carga se haga correctamente. Si el tipo de nuestra tarjeta no existe en el software, simplemente añádalo utilizando el administrador de tarjetas (Herramientas > Tipo de tarjeta > Administrador de tarjetas).

Funciones

Como en cualquier lenguaje de programación, el sistema de funciones también está presente en el lenguaje Arduino. Todas las funciones y operadores básicos como «if», «else», «switch case», «==», «!=», etc. son comunes a muchos lenguajes, y por lo tanto no se presentarán en este artículo. Si no estás familiarizado con estas funciones, te aconsejamos que empieces tu aprendizaje con un tutorial de lenguaje C.

Shares



Función de gestión de E/S digitales

- **pinMode** (spindle, mode): esta función se utiliza para definir un cabezal específico del Arduino (por su número) como entrada o salida.
- **digitalWrite**(huso, valor): se utiliza para definir el estado de una salida en un momento dado. Por ejemplo, si has definido que un LED está conectado al pin 4 (`pinMode(4, OUTPUT);`), puede enviar energía a este pin, tanto si se enciende como si no, por ejemplo:
 - `digitalWrite(4, HIGH)` ; se utiliza para enviar una señal eléctrica y, por lo tanto, para encender el LED.
 - `digitalWrite(4, LOW)` ; se utiliza para conectar a tierra el pin 4 al potencial de tierra y, por lo tanto, para apagar el LED.
- **int digitalRead**(pin): se utiliza para leer el estado de un pin. Si se conecta a un potencial de 5V, el resultado de la lectura será 1, y si se conecta a un potencial cero, el resultado será 0.

Gracias a estas funciones y a algunos componentes electrónicos, es posible, por ejemplo, **encender lámparas, accionar motores o comprobar el estado de un elemento conectado a un sensor leyendo su estado.**

Función de gestión de E/S analógicas

Muchos Arduino tienen entradas analógicas, pero muy pocos (como Due) tienen salidas analógicas.

- **int analogRead**(pin): la función `analogRead` permite leer el estado de un pin analógico. La diferencia entre un pin analógico y un pin digital es que a diferencia de un pin digital que devuelve sólo un 1 o 0, el pin analógico envía un número entre 1 y 256, siendo el 1 0V y 256 5V. Por lo tanto, podemos tener un número que corresponde a una tensión, y por lo tanto conocer el potencial de un cabezal en un momento 'T'. También puedes aumentar la precisión del Arduino para obtener más precisión. Las entradas analógicas se utilizan ampliamente para sensores que devuelven una multitud de valores diferentes, como por ejemplo, sensores de temperatura.
- **analogWrite**(pin, valor): pocas tarjetas tienen esta función. Se utiliza para enviar una tensión proporcional a un delta, que puede modificarse si es necesario para aumentar la precisión.

Funciones de temporización

Puesto que Arduino tiene un reloj, es posible realizar retardos de tiempo entre diferentes bloques de código. Aquí están las funciones que le permiten manejar el tiempo:

Shares



1. **Delay()** : La función Delay(ms) se utiliza para hacer una pausa de una cierta duración establecida por un número entero pasado en la función cuando se llama. Este número entero corresponde al número de milisegundos durante los cuales se debe parar el programa. Esta función es una función de bloqueo y, por lo tanto, mientras se ejecuta, se ignoran todas las demás acciones.
2. **DelayMicrosegundos()**: Igual que la función delay(), excepto que el entero pasado en el parámetro se tiene en cuenta como microsegundos y no como milisegundos.
3. **Unsigned long millis()**: permite devolver un número de milisegundos que corresponde al tiempo transcurrido desde el inicio del programa. Esta función se reajusta después de unos 50 días después de un desbordamiento.
4. **Unsigned long Micro()**: exactamente la misma función que millis(), excepto que micro() devuelve un entero que corresponde a microsegundos y ya no a milisegundos.
5. **Long random(max)**: devuelve un número entero aleatorio entre 0 y el número pasado en el parámetro.
6. **Long random(min, max)**: devuelve un número entero aleatorio entre los dos números pasados en los parámetros.

Función Scheduler

La función del programador es una de las funciones más importantes de Arduino, ya que nos permite realizar tareas asíncronas. De hecho, básicamente, un Arduino no puede manejar varias tareas al mismo tiempo. Por ejemplo, si usas la función delay() para hacer parpadear un LED, y quieres leer el estado de una entrada de Arduino al mismo tiempo, no podrás hacerlo, porque mientras el delay se está acabando, el programa está bloqueado, y está esperando a que termine.

Como vimos al principio, el programa está compuesto sólo por dos funciones: «setup()» y «loop()». Por supuesto, es posible crear otras funciones, pero se ejecutarán sincrónicamente después de su llamada. La función planificador de procesos< nos permite crear varios bucles infinitos (idénticos a la función de bucle existente) y lanzarlos al mismo tiempo. Un ejemplo del uso de esta función para trabajar diferentes tareas:

```
// Include Scheduler since we want to manage multiple
tasks.
#include

int led1 = 13;
int led2 = 12;
int led3 = 11;

void setup() {
  Serial.begin(9600);
```

```
// Setup the 3 pins as OUTPUT
pinMode(led1, OUTPUT);
pinMode(led2, OUTPUT);
pinMode(led3, OUTPUT);
```

Shares



```
// Add "loop2" and "loop3" to scheduling.
```



```
// "loop" is always started by default.
```



```
Scheduler.startLoop(loop2);
```



```
Scheduler.startLoop(loop3);
```



```
}
```



```
// Task no.1: blink LED with 1 second delay.
```

```
void loop() {
```

```
digitalWrite(led1, HIGH);
```

```
// IMPORTANT:
```

```
// When multiple tasks are running 'delay' passes control
to
```

```
// other tasks while waiting and guarantees they get
executed.
```

```
delay(1000);
```

```
digitalWrite(led1, LOW);
```

```
delay(1000);
```

```
}
```

```
// Task no.2: blink LED with 0.1 second delay.
```

```
void loop2() {
```

```
digitalWrite(led2, HIGH);
```

```
delay(100);
```

```
digitalWrite(led2, LOW);
```

```
delay(100);
```

```
}
```

```
// Task no.3: accept commands from Serial port
```

```
// '0' turns off LED
```

```
// '1' turns on LED
```

```
void loop3() {
```

```
if (Serial.available()) {
```

```
char c = Serial.read();
```

```
if (c=='0') {
```

```
digitalWrite(led3, LOW);
```

```
Serial.println("Led turned off!");
```

```
}
```

```
if (c=='1') {
```

```
digitalWrite(led3, HIGH);
```

```
Serial.println("Led turned on!");
```

```
}
```

```
}
```

```
// IMPORTANT:
// We must call 'yield' at a regular basis to pass
// control to other tasks.

yield();
}
```

Shares



Puesto que el microprocesador de Arduino no está diseñado para hacer asíncronos, esta función le permite hacerlo como hemos visto anteriormente, pero de una forma un tanto indirecta. De hecho, cuando se usa la función de programar, el Arduino no ejecuta ambos bucles al mismo tiempo; el primer bucle ejecutará una parte muy pequeña de su código, pasará el relé al segundo bucle que también ejecutará una parte del código, luego pasará la mano al primer bucle, y así sucesivamente. En general, en el caso de los proyectos pequeños, esto no es en absoluto perturbador. Sin embargo, para aquellos que quieran ejecutar código en un tiempo específico, esta función debe ser evitada, ya que cambiar de un bucle a otro aumenta el tiempo de cálculo de Arduino.

Además, es imprescindible utilizar la función «yield()» al final de cada bucle, ya que es la que permite gestionar la transición de uno a otro.

Funciones personalizadas

Por supuesto, también podemos crear funciones personalizadas que satisfagan nuestras necesidades. En lugar de códigos largos, aquí hay un ejemplo de un código que te permite hacer que un LED parpadee un número específico de veces al presionar un botón, el número de repeticiones se especifica durante la llamada:

```
/*
 * Impulsos programados
 * -----
 * Cambia de estado ON Off un LED conectado en el una
 salida digital
 * pin 13. El se encenderá y apagará tantas veces como se
 indique mediante un digito
 * ASCII leído desde el puerto serie. * * Created 18
 October 2006
 * copyleft 2006 Tod E. Kurt <tod@todbot.com>
 * http://todbot.com/
 *
 * based on "serial_read_advanced" example
 */
int ledPin = 13;
// selecciona el pin para el LED
int val = 0;
// variable que almacena el valor leído del puerto
void setup() { pinMode(ledPin,OUTPUT);
// declara el PIN del LED como salida
Serial.begin(19200);
```

```
// conecta con el puerto serie a la velocidad de 19200
Serial.println("Bienvenido al Generador de Impulsos
Programados"); } void loop () { val = Serial.read();
// lee el numero del puerto (una sola cifra)
```

Shares



//si el valor leído es un solo digito se ejecuta el programa



El uso de funciones personalizadas es idéntico al de los lenguajes C y C++.



Monitor en serie

Gracias al serial monitor (el pequeño icono de la lupa en la parte superior derecha) tienes la posibilidad de mostrar información del código. Esto se usa a menudo para depurar, o simplemente para registrar información devuelta por el Arduino, en vivo. En el código que escribimos, podemos especificar al Arduino que queremos mostrar cierta información, o variables. Aquí hay un ejemplo:

```
/*

DigitalReadSerial

Reads a digital input on pin 2, prints the result to the
serial monitor

This example code is in the public domain.

*/

// digital pin 2 has a pushbutton attached to it. Give it
a name:
int pushButton = 2;

// the setup routine runs once when you press reset:
void setup() {
// initialize serial communication at 9600 bits per
second:
Serial.begin(9600);
// make the pushbutton's pin an input:
pinMode(pushButton, INPUT);
}

// the loop routine runs over and over again forever:
void loop() {
// read the input pin:
int buttonState = digitalRead(pushButton);
// print out the state of the button:
Serial.println(buttonState);
delay(1); // delay in between reads for stability
}
```

Este programa nos permite ver el estado de un botón en vivo. Cuando se encuentra en la ventana del monitor serie y el programa se inicia en el Arduino, puede ver las palabras «Press button» cuando se pulsa.

La línea «Serial.begin(9600)» en la configuración es obligatoria, y se utiliza para inicializar el enlace serie de Arduino, asignándole una velocidad de transmisión. Esta velocidad puede, por supuesto, ser cambiada, pero también será necesario modificar este valor en el monitor serial para que la información sea leída en la misma frecuencia que su transmisión.

Shares



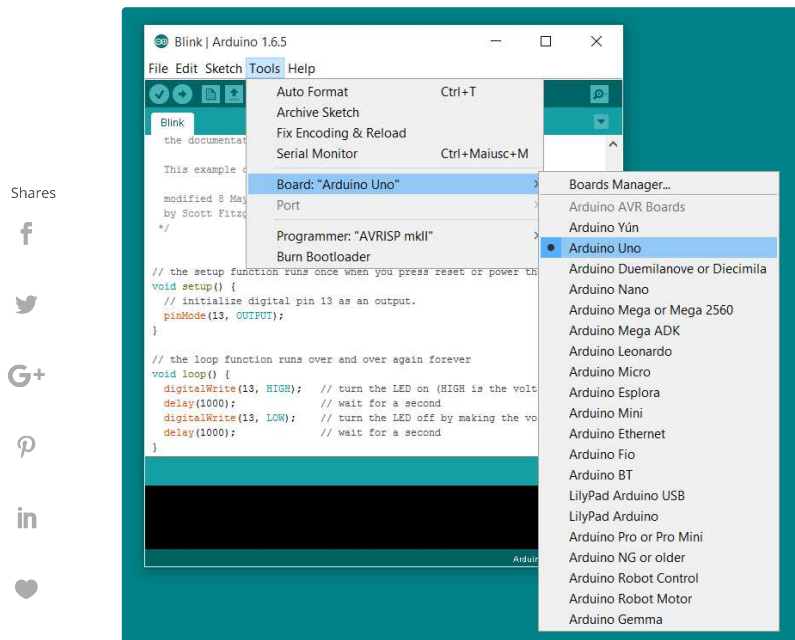
Librerías Arduino

Las librerías son colecciones de funciones que permiten controlar distintos dispositivos. Estas son algunas de las librerías más utilizados:

- librería GPS
- LCD
- Servo
- SD
- Robot_control
- Robot_motor
- Ethernet
- Wi-Fi
- Stepper
- SPI
- [EEPROM](#)
- Software Serial
- GSM

Pasos para configurar la placa

1. En primer lugar, se instala el software IDE. Se puede descargar el IDE desde [el sitio web oficial](#).
2. Instalar el software en su PC.
3. Ahora ejecutar el archivo .exe IDE Arduino. Tiene un siguiente distribución:
4. Crear el programa usando el editor de código y cargarlo en la placa. Para ello, es necesario conectar la placa al ordenador mediante un cable USB.
5. En el IDE, hay que seleccionar el tipo de placa que estás utilizando de las Herramientas -> menú de placas.
6. Ahora comprueba el código haciendo clic en el icono de comprobación en la parte superior de la ventana del IDE, a continuación, haz clic en el botón de compilar y carga el código en la placa.



Esó sí quizás tengas que instalar los controladores si el sistema no detecta nuestra placa.

Shields o módulos

Los shield o módulos son tableros o módulos que se conectan a la placa y que permiten conectar dispositivos periféricos, sensores y otros dispositivos nuestro proyecto. A continuación os listamos los mas populares:

- shield GSM
- shield Ethernet
- shield WiFi
- shield Motor
- Proto shield
- Joystick
- shield Bluetooth
- shield Xbee

Componentes y Accesorios

A continuación os mostramos una lista de todos los componentes y accesorios de uso general para usar estas placas a la hora de desarrollar proyectos:

- Breadboard
- Cable USB
- Batería de 9V
- LEDs
- Pulsadores o botones
- Condensadores
- Reguladores de voltaje
- Sensores (IR, de temperatura, etc.)
- Puentes
- Resistencias
- Potenciómetros

- Motors (DC, Servo, BLDC)
- LCD
- Teclado Hex
- Modulos o shields

Shares

f Enlaces de interes

- 🐦 [Arduino Qué es, cómo empezar y dar los primeros pasos](#)
- G+ [Elegir la placa Arduino adecuada para tu proyecto. Una introducción.](#)
- p [5 consejos a la hora de comprar y empezar con Arduino](#)
- in [Análisis del Arduino Starter Kit](#)
- ♥ [Foro Arduino](#)

[amazon box=»B01M7ZB2B4″]

Si estás empezando y no te atreves a meterte a programar con el código te recomendamos que empieces usando una alternativa gráfica para la programación, en [este artículo](#) te enseñamos unas cuantas.

Aquí os dejamos un podcast donde nos dan brevemente una introducción a Arduino.



Por **JorgeRubira**

JavaHispano Podcast - 132 - Introducción a Arduino (Entrevista a BricoGeek)

3

[Descargar](#) | [Suscribirse](#) | [Descargar APP](#)

REPRODUCIR

00:00 29:05

Unos cuantos proyectos para empezar:

[10 proyectos arduino sencillos para principiantes](#)

[5 proyectos para empezar con Arduino este verano](#)

[2 divertidos proyectos arduino funcionando con energía solar](#)

[2 proyectos de Arduino para ayudarnos en la cocina](#)

[5 proyectos divertidos para hacer con Arduino y LEGO](#)

Si te has quedado con más ganas de proyectos realizados con Arduino pásate por [la sección de proyectos con Arduino de nuestra web](#).

Arduino es una marca registrada por la
compañía Arduino.cc. Raspberry Pi es una
marca registrada de The Raspberry Pi

Fundation

Este sitio es no tiene nada que ver con las anteriores marcas, solo pretendemos difundir el Hardware Libre. Para poder funcionar Descubrearduino.com participa en el Programa de Asociados de Amazon Services LLC, un programa de publicidad de afiliados diseñado para proporcionar un medio para que los sitios ganen tarifas de publicidad mediante la publicidad y los enlaces a Amazon.com. Este sitio también participa en otros programas de afiliados y es compensado por referir tráfico y negocios a estas compañías.

Shares

[Contacta con nosotros](#)[Política de privacidad](#)

Descubrearduino.com Noticias de Hardware libre desde 2014
Web diseñada y optimizada por [TuEquipoSeo](#) Para
Descubrearduino.com